SIEM Deployment Guide (Wazuh)

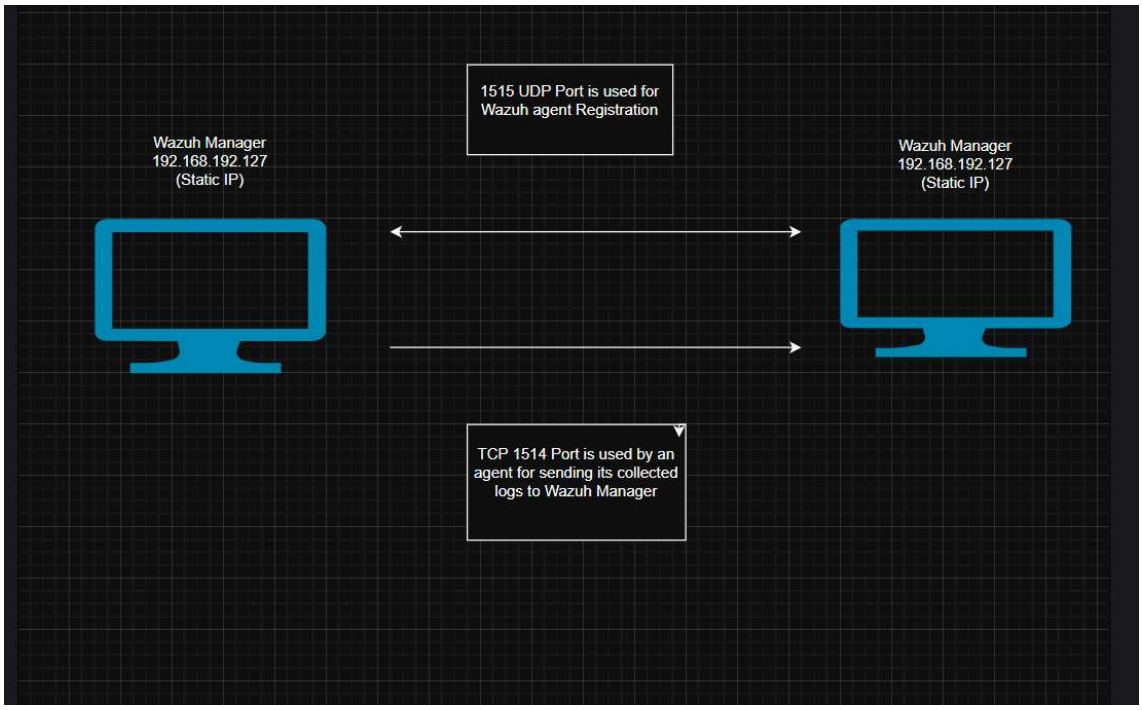
– Description : -

- This project demonstrates the setup of a Security Monitoring environment using **Wazuh** (Manager, Agent, and Dashboard) integrated with **Suricata IDS**.
- The goal is to detect and visualize network intrusion attempts and custom Suricata alerts within the Wazuh Dashboard.

– Environment Setup : -

The environment consists of :

- Wazuh manager installed on Ubuntu Server (e.g., IP: 192.168.192.127)
- Wazuh dasboard installed on the same system as the manager
- Wazuh agent installed on a monitored endpoint system
- Suricata IDS installed on the agent machine to monitor traffic and generate alerts



– Installing and configuring Wazuh manager, Agent and Dashboard : -

- The Wazuh Manager is responsible for collecting, analyzing, and storing security events from all agents.
- After installation, the Wazuh Dashboard was accessed via the web interface at: https://192.168.192.127/
- The dashboard provides an overview of agent status and alerts.

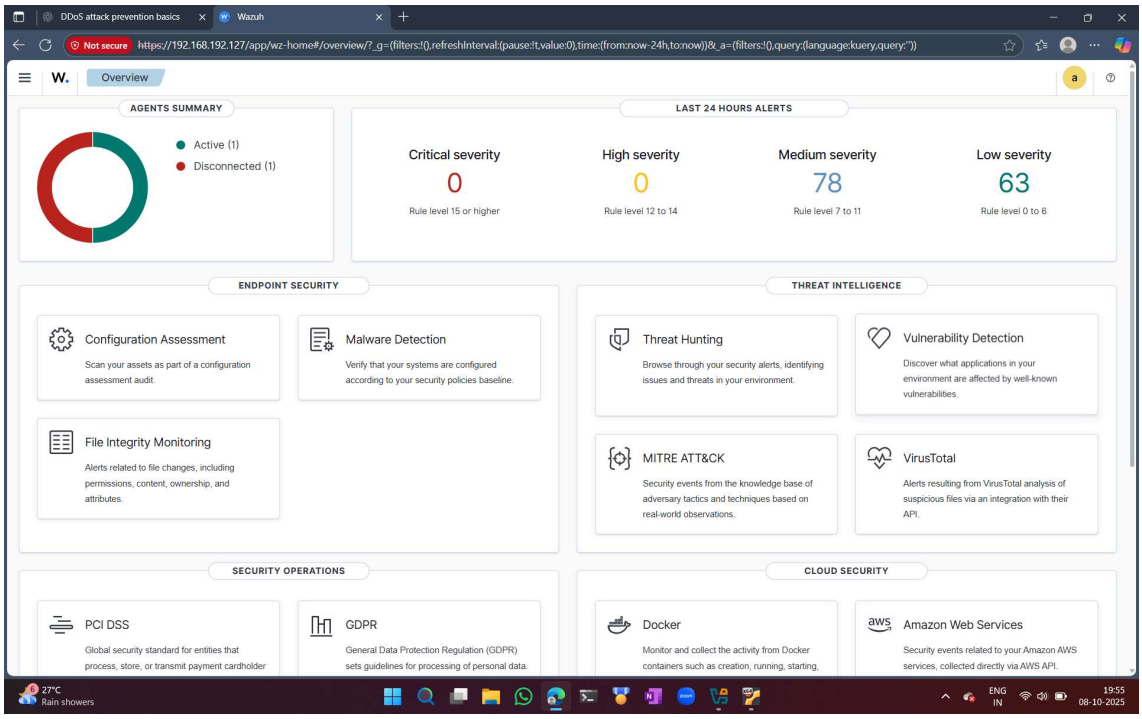


Figure 1 — Wazuh Dashboard Overview showing connected agents and alert summary.

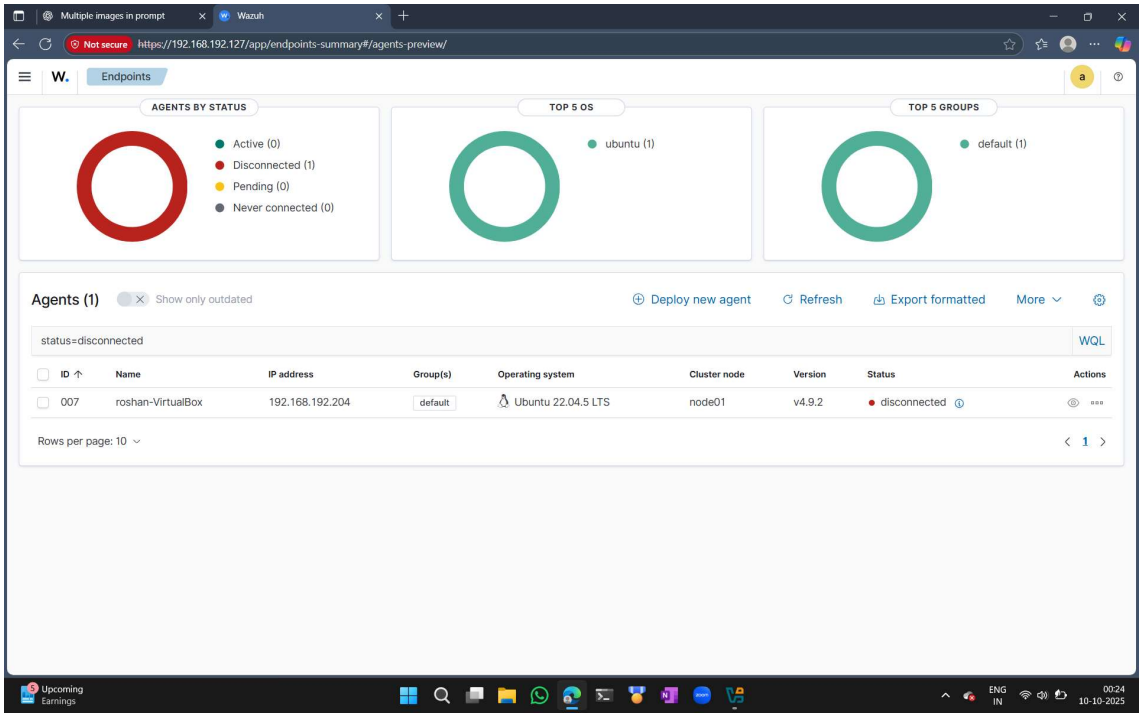


Figure 2 — Agent summary showing connected Wazuh agent.

- The Wazuh Agent was installed on a monitored endpoint and successfully connected to the Wazuh Manager.
- This screenshot shows one active agent that was disconnected agent in the system.

– Installing and Configuring Suricata : -

- Suricata was installed on the endpoint where the Wazuh agent is running.
- It was configured to monitor network traffic and generate alerts stored in eve.json.
- To view or read the logs in install ‘jq’
- The configuration file used: /etc/suricata/suricata.yaml
- We can also enable Suricata IPS in the agent by writing the Suricata dropping or blocking rules.

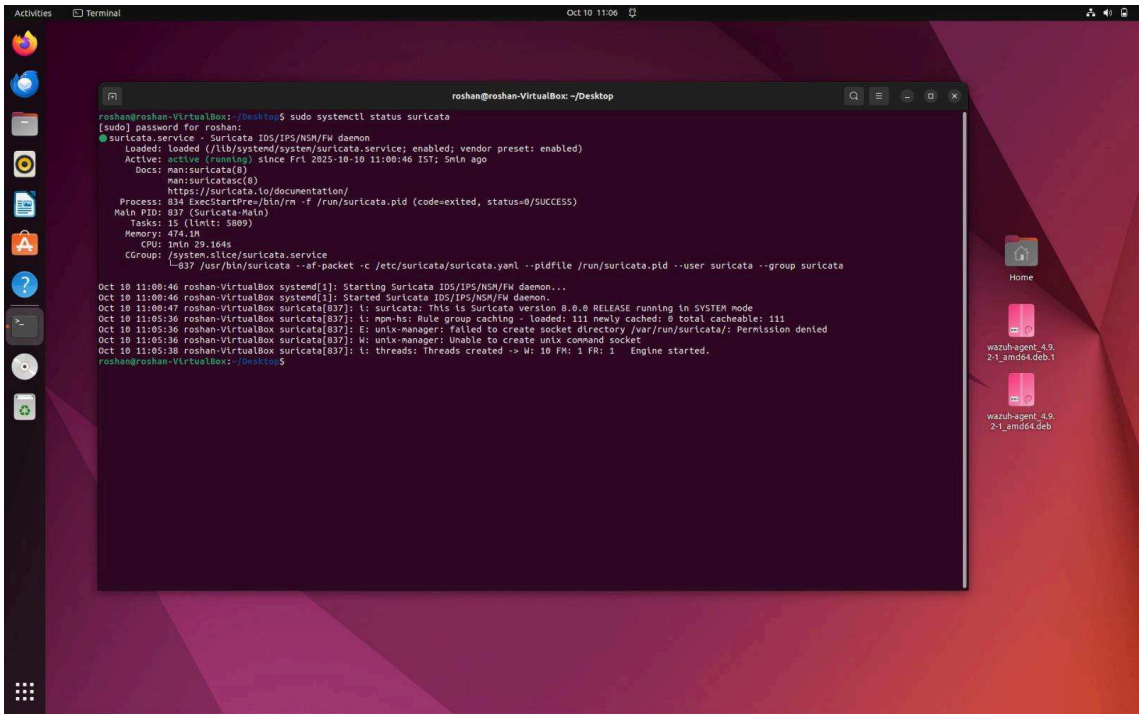


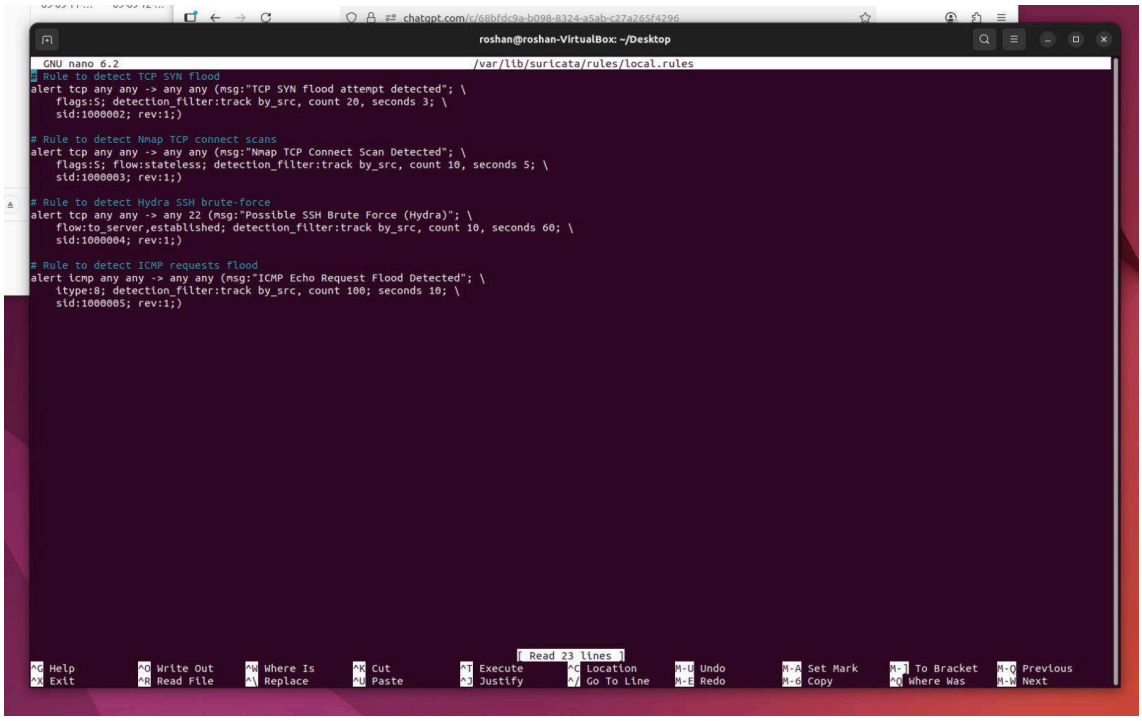
Figure 3 — Suricata installation and service status.

– Writing and Loading Custom Suricata rules : -

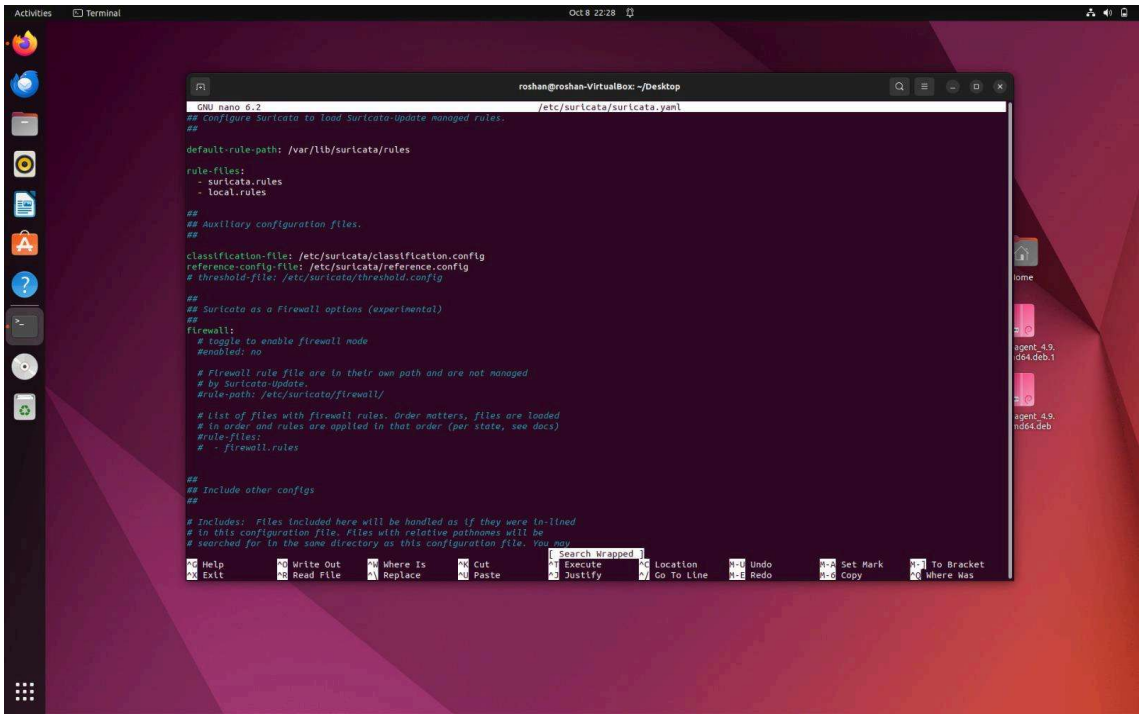
- Custom Suricata rules were written inside /etc/suricata/rules/local.rules.
- These rules are designed to detect specific test attacks (e.g., ICMP, SSH brute force, HTTP SQL injection, etc.).
- **Example custom rule :**

alert icmp any any -> any any (msg:"ICMP Echo Request Detected"; sid:1000001; rev:1;)

- The rule file was configured under the rule-files section of /etc/suricata/suricata.yaml for Suricata to load it.



- These are the Suricata Custom rules that has to be written only in the file /etc/suricata/rules/local.rules
- The Suricata default in-built rules are stored in the directory /var/lib/suricata/rules This directory usually contains the default/system-installed Suricata rules, often from the community rulesets like Emerging threats.
- After writing the Suricata custom rules, run **‘sudo suricata -T -c /etc/suricata/suricata.yaml’** to load the Suricata custom rules.
- In most cases Suricata drop rules are not used since mostly false-positives will arrive.
- All Suricata custom rules are to be only in one format either single line or double lines if not means Suricata will not detect the attacks.
- The **‘sid’** value for Suricata custom rules will start after the 100000.
- The **‘count’** and **‘seconds’** parameters in the rule is number of packets arriving within a specified timeframe called seconds to the system.



- Here the default rule-path given is /var/lib/suricata/rules in the suricata configuration file /etc/suricata/suricata.yaml
- Here Suricata does not load the suricata custom rules because custom rules file path /etc/suricata/rules/local.rules is not configured under the rule-files section in the suricata configuration file.
- If we write the suricata custom rules in the file /var/lib/suricata/rules/local.rules it may cause overwritten problem with the file /var/lib/suricata/rules/local.rules when we run suricata-update tool.
- When you run sudo suricata-update, Suricata fetches new rules and may recreate or replace /var/lib/suricata/rules/local.rules with a blank or default version.
- **Issues caused during overwritten problem :**

– Custom rules lost **-** your manually created rules (e.g., ICMP, SSH brute force) get replaced or deleted.

– Alerts stop generating **-** since suricata no longer loads your custom rules, you stop getting alerts for your test attacks.

- So, to overcome this we may “Symbolic link” which is also known as symlink between the files /var/lib/suricata/rules/local.rules and /etc/suricata/rules/local.rules
- A symlink is like a *shortcut* or *pointer* in Linux.
- It links one file or directory to another — so if a program looks for a file in one location, it can actually be read from another location transparently.
- Otherwise, configure the direct suricata custom rules file as /etc/suricata/rules/lcoal.rules under the rule-files section in the suricata configuration file suircata.yaml

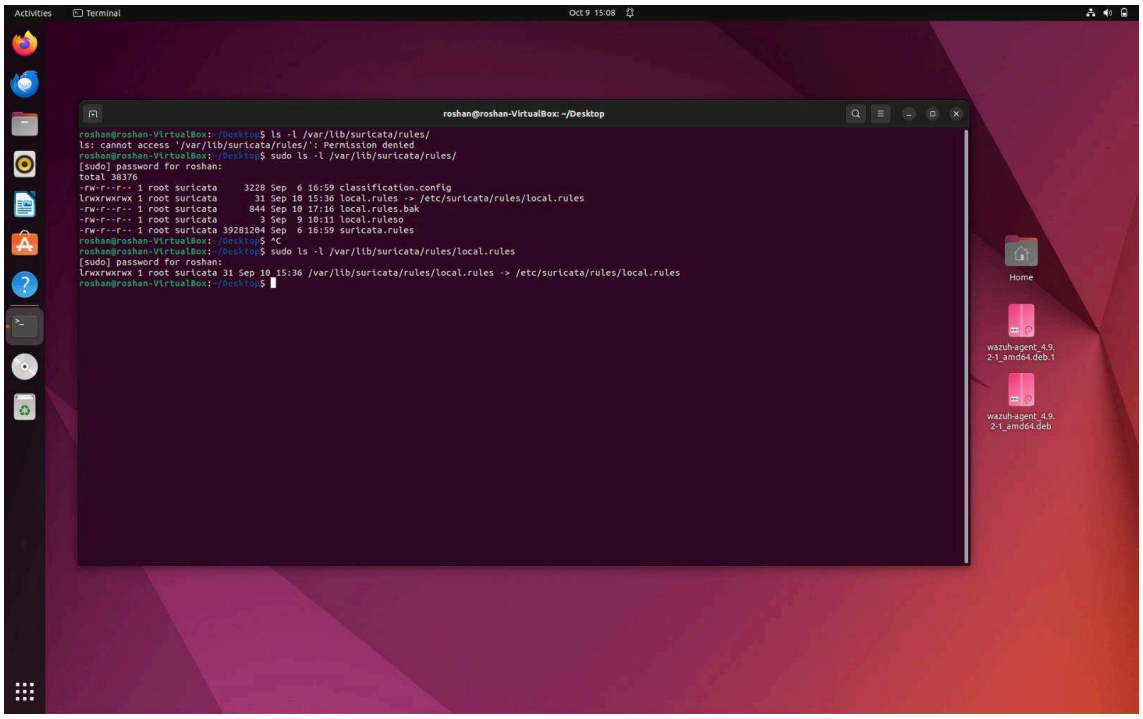


Figure 5 — Symlink showing link between default and custom rule directories.

– Triggering and Capturing Alerts : -

- To test the Suricata setup, sample attacks were simulated (such as ICMP ping or TCP port scans) against the monitored host.
- These actions triggered alerts that were logged in /var/log/suricata/eve.json.
- To read the logs in human readable format we install the tool ‘jq’.

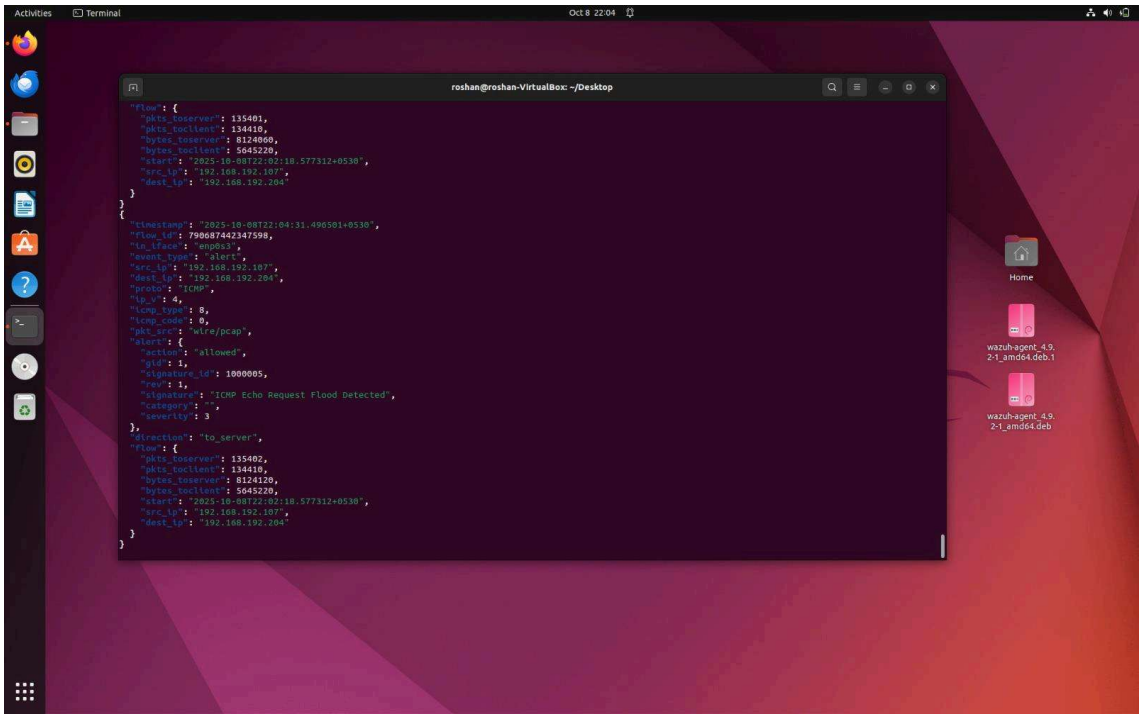


Figure 6 — Test attack traffic generated to trigger Suricata rule.

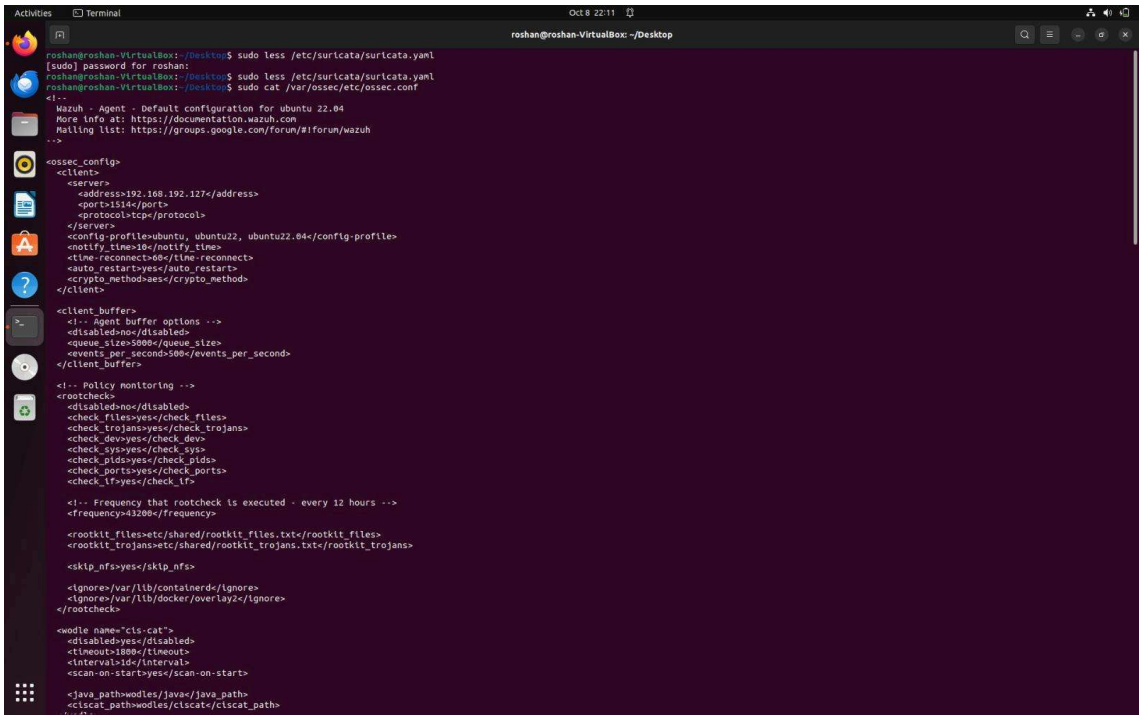


Figure 7 — Wauzh agent Configuration file send lpgs to the Wazuh Manager IP Address 192.168.192.127

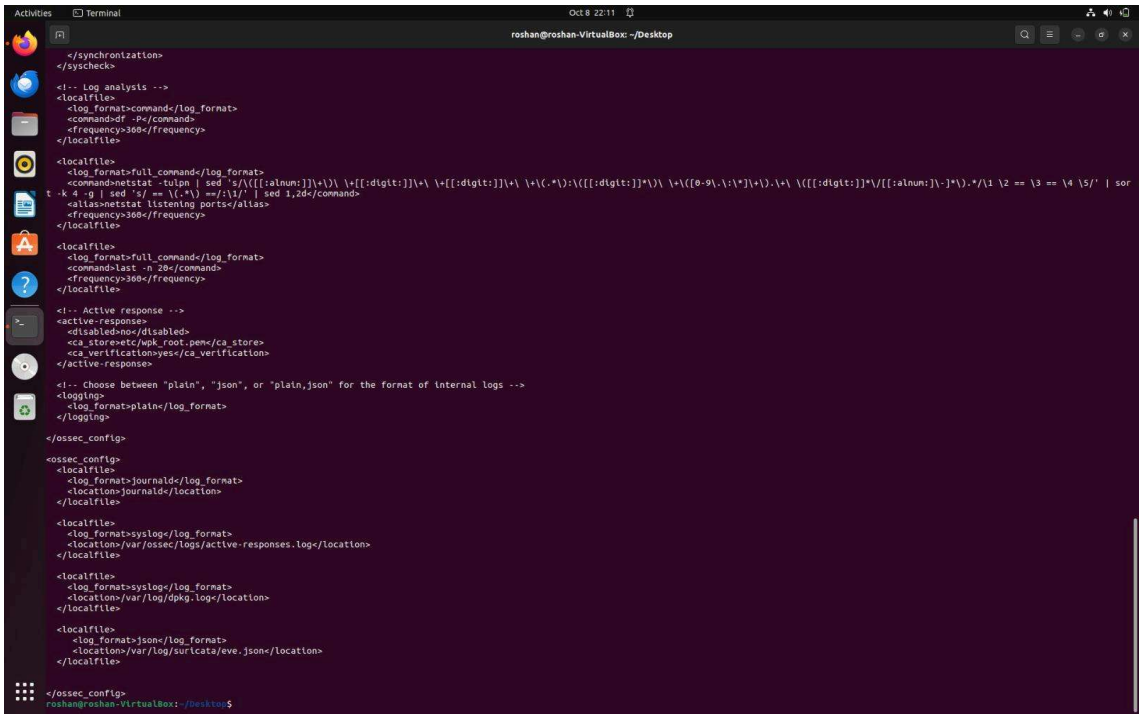


Figure 8 — Wazuh agent stores different formats of logs in different file locations

– Viewing Alerts in Wazuh Dashboard : -

- The alerts generated by Suricata were successfully forwarded to the Wazuh Manager via the Wazuh Agent.
- Wazuh parsed the Suricata logs and displayed the corresponding alerts in the Dashboard under *Security events → Suricata*.
- The alert contains fields such as timestamp, source/destination IPs, protocol, and custom message.

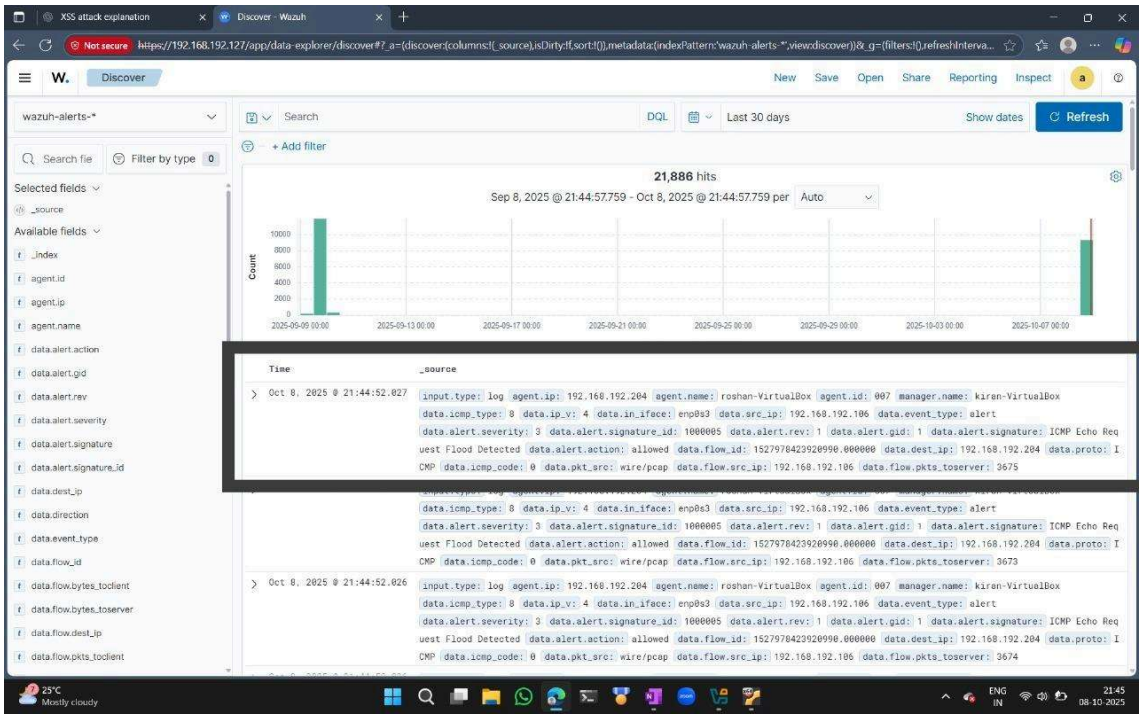


Figure 9 — ICMP Flood Suricata rule alerts successfully displayed in Wazuh Dashboard.

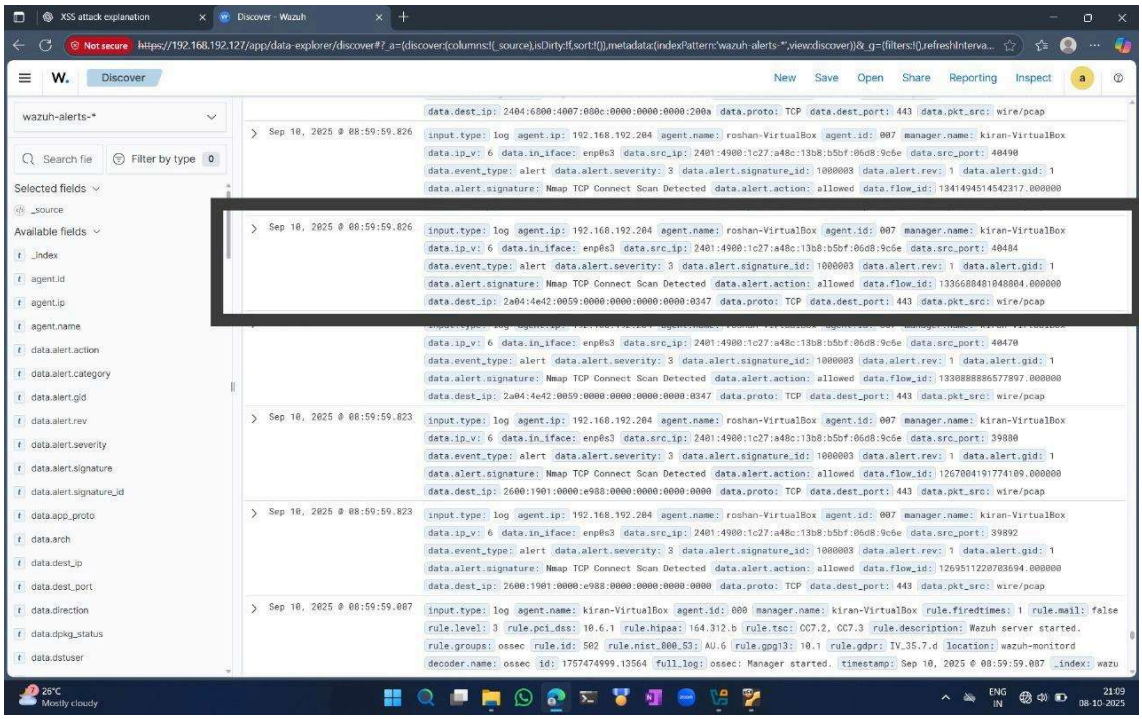


Figure 10 — Nmap Scan detection Suricata rule alerts successfully displayed in Wazuh Dashboard.

– Understanding Alerts Severities : -

Wazuh categorizes alerts based on rule levels **(0–15)** :

- 0–6 → Low severity
- 7–11 → Medium severity
- 12–14 → High severity
- 15+ → Critical severity

The dashboard displays alert counts by severity level over the last 24 hours.

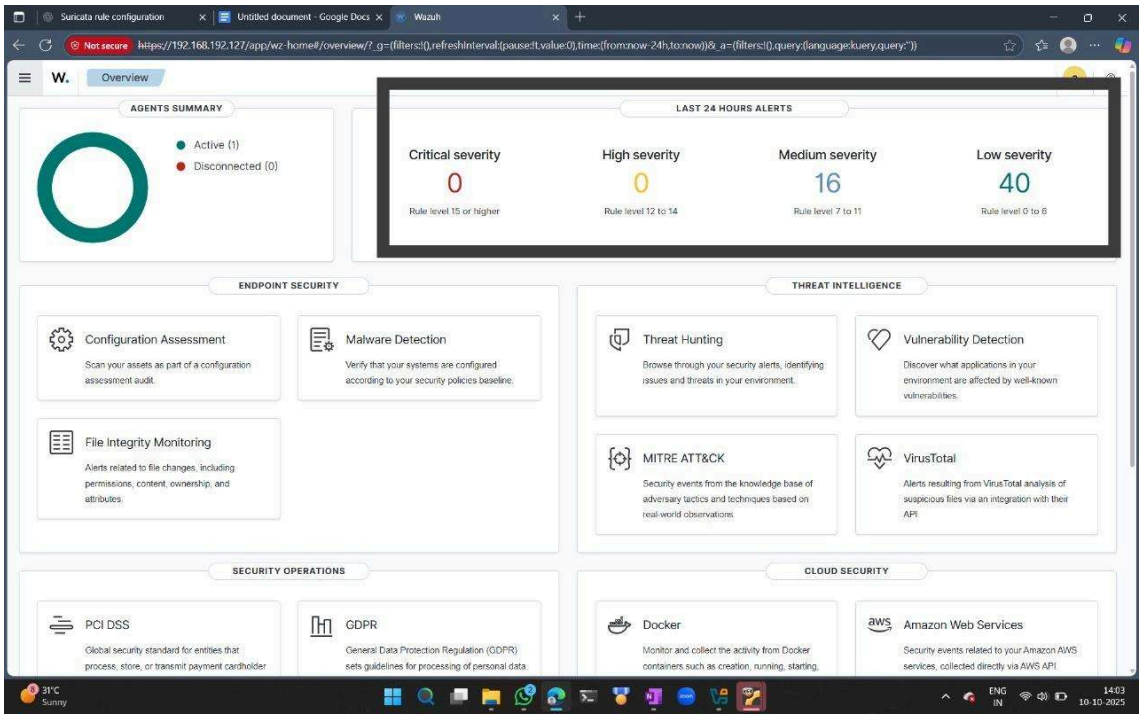


Figure 11 — Severity distribution of alerts in the Wazuh Dashboard.

– Threat Hunting and MITRE ATT&CK Mapping : -

- Wazuh integrates with **MITRE ATT&CK** to classify detected events based on known adversary tactics and techniques.
- This helps security analysts understand the context and purpose of detected behaviors.

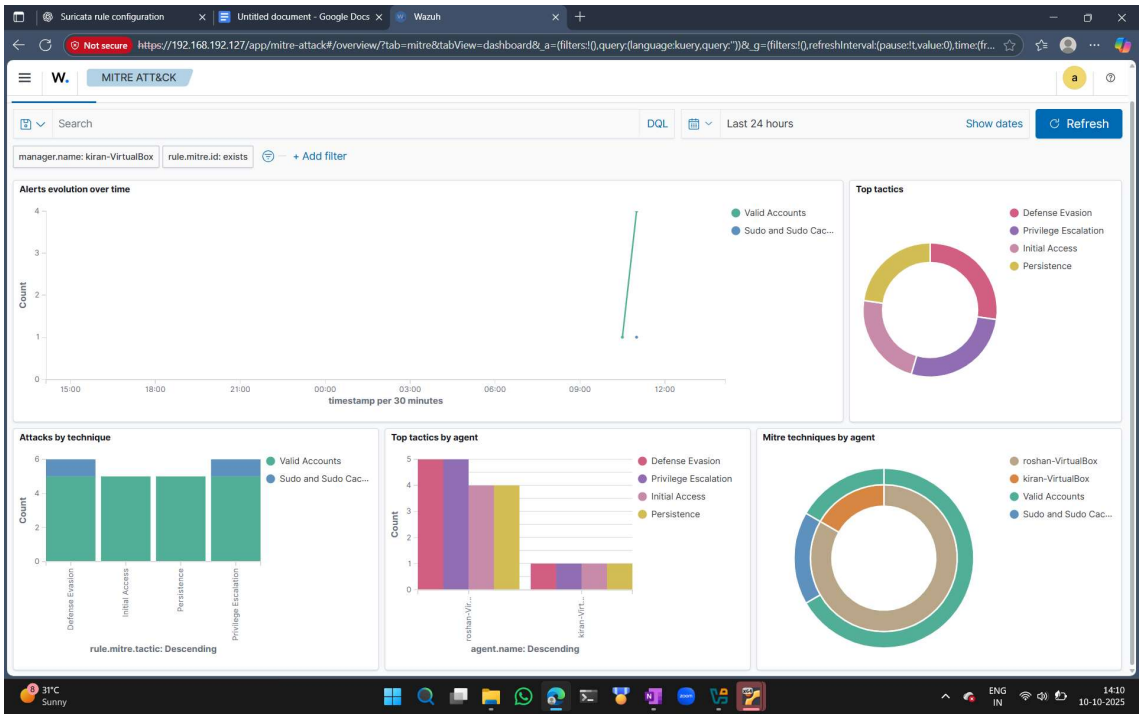


Figure 12 — MITRE ATT&CK mapping of detected threats in Wazuh.

– File Integrity Module (FIM) Testing : -

- Initially, create a test file named “fimtest.txt”
- Open the Wazuh agent Configuration File sudo nano /var/ossec/etc/ossec.conf and configure the full file path of the file “fimtest.txt” under the section <syscheck>
- The wazuh agent automatically calculates the hash value (MD5, SHA1, SHA256 etc.) of that file “fimtest.txt” and stores the hash value, some metadata.
- So if the File is modified the hash value changes then the wazuh agent raises an alert to the wazuh manager.
- We can see that alert in File Integrity Module (FIM) in the wazuh dashboard.

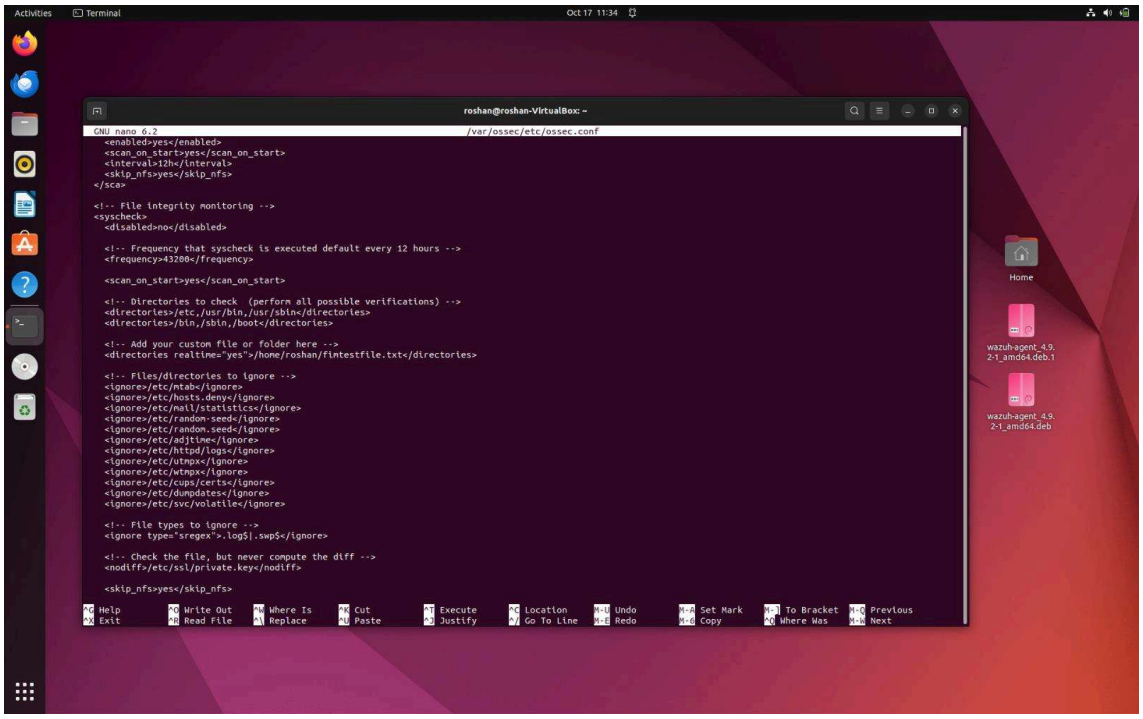


Figure 13 — Configuring the File path in wazuh agent configuration file.

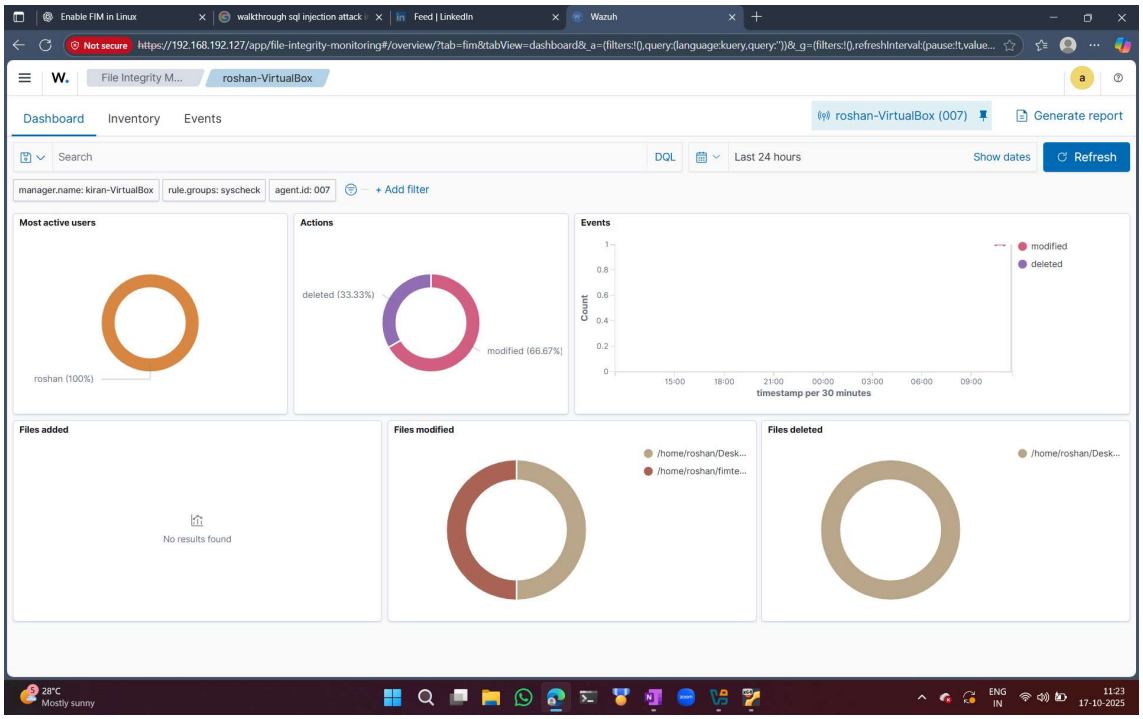


Figure 14 — Wazuh dashboard displays what files are modified and deleted in the File Integrity Module (FIM) section.

– Conclusion : -

- This project demonstrated how Wazuh and Suricata can be integrated for real-time intrusion detection and alert visualization.
- By creating and loading custom Suricata rules, network attacks were successfully detected and displayed on the Wazuh Dashboard.
- The setup provides a complete open-source SIEM + IDS monitoring environment.
- The deployment of Wazuh as the central SIEM platform successfully establishes a robust, scalable, and security-focused monitoring framework for the organization.
- Through centralized log collection, real-time threat detection, file integrity monitoring, and automated alerting, Wazuh enhances the organization's visibility across endpoints, servers, network devices, and cloud workloads.
- By integrating Wazuh with external tools such as Suricata, TheHive, Cortex, and threat-intelligence sources, the environment now supports end-to-end security operations—covering detection, enrichment, investigation, and response. The implementation also enables structured incident workflows, faster triage, and improved threat-hunting capabilities through its correlation rules and comprehensive dashboards.